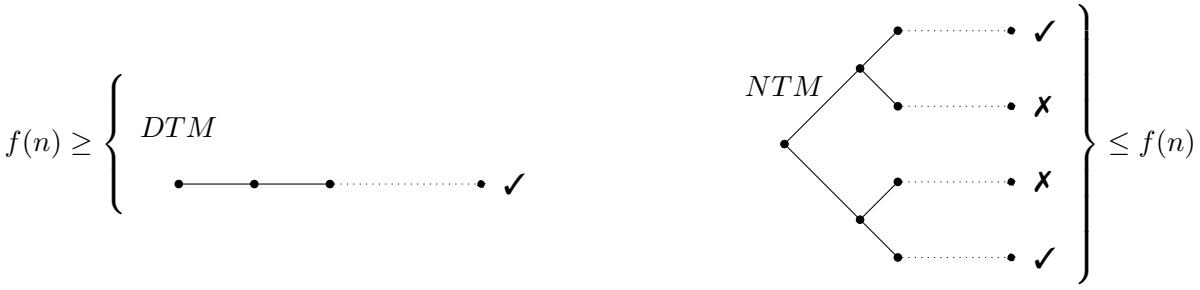
1 Complexity Theory

What are the problems that can or cannot be solved **efficiently**?

Definition The running time, or time complexity of a deterministic Turing Machine (DTM) M is a function $f : \mathbb{N} \to \mathbb{N}$, where $f(n)$ is the maximum number of steps M takes on any input of length n.

Definition The time complexity of a non-deterministic Turing Machine (NTM) M is a function $f : \mathbb{N} \to \mathbb{N}$, such that any branch of computation of M on any input of size n takes a maximum of $f(n)$ steps.



Remark It actually suffices to consider only the accepting paths when measuring the time complexity $f(n)$ since any computation that takes longer than $f(n)$ can safely be rejected.

Definition P is the class of languages that are accepted by some polynomial-time DTM.

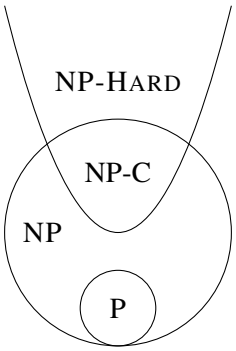
Definition NP (**non-deterministically polynomial**) is the class of languages that are accepted by some polynomial-time NTM.

2 P vs NP

Is P = NP?

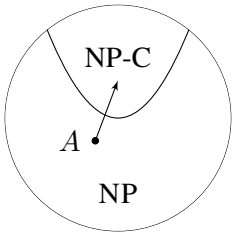
3 NP-Completeness

The "hardest" problems of NP.



Definition A language L is NP-COMPLETE if:

- $L \in$ NP, and
- Every language in NP can be reduced to L through a poly-time reduction, i.e. L is NP-HARD. ($A \leq_P L, \forall A \in$NP)



Remark If L is NP-COMPLETE, and L has a poly-time solution ($L \in$ P), then P = NP.

Remark If L is NP-COMPLETE, and $L \leq_P L'$ for some $L' \in$NP, then L' is NP-COMPLETE.

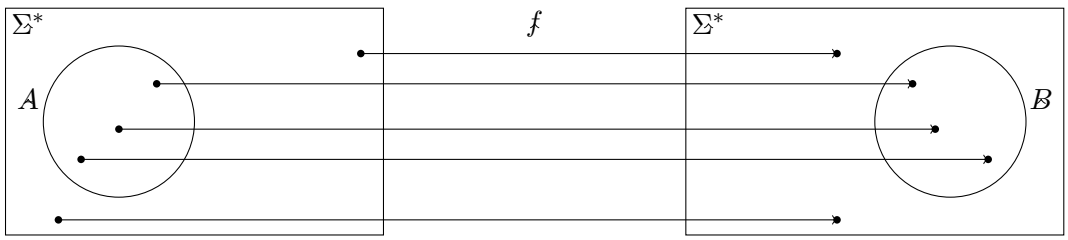
4 Polynomial-Time Reduction

Definition A "computable function" $f : \Sigma^* \to \Sigma^*$ is a function such that a TM M on input w halts with $f(w)$ on its tape.

Definition A function f is "polynomial-time computable" if it is computable by some poly-time DTM.

Definition Language A is "polynomial-time reducible" to language B ($A \leq_P B$) if a poly-time computable function $f : \Sigma^* \to \Sigma^*$ exists such that, $\forall w \in \Sigma^*$,

$$w \in A \iff f(w) \in B$$



"Mapping Reducible"
A $\leq_P$ B

Remark Our previous reduction from E_{TM} to EQ_{TM} is a mapping reduction.

5 Additional remarks and definitions

5.1 Decision vs. Optimization

- P, NP deal with decidable languages (i.e., decision problems).
- But more general problems (e.g., optimization) usually have some decision version. For example, "what is the shortest path between x and y?" vs. "is there a path between x and y shorter than n?"
- If the decision version is "intractable" (problems that can be solved in principle, but require so many resources that they are practically impossible to solve for large inputs), then certainly so is the optimization version.

5.2 Encodings and "size of the input"

- An integer is assumed to be encoded in $O(\log n)$ characters.
- A graph $G = (V, E)$ can be encoded in $O(E \log V)$ characters.

6 The first NP-COMPLETE language

SAT= $\{<\Phi> \mid \Phi$ is a Boolean formula that is satisfiable$\}$

Remark A satisfiable Boolean formula is a Boolean formula that has a satisfying assignment (i.e., that evaluates to 1).

Example $\Phi = (\overline{x} \wedge y) \vee (x \wedge \overline{z})$ is satisfiable.
E.g., $x = 0, y = 1, z = 1$, or $x = 1, y = 1, z = 0$

Example $\Phi = \overline{x} \wedge x$ is **not** satisfiable.

Theorem 6.1 *Cook-Levin Theorem, 1971:* SAT *is* NP-COMPLETE.

Proof Proof has two parts:

1. SAT $\in$ NP.
   An NTM can test every possible assignment, or a DTM can verify a possible solution.

   The NTM Approach (The Searcher) This approach focuses on the "computational power" of nondeterminism. For each variable x_i in the formula, the NTM "branches." One branch assumes $x_i = 1$ and the other assumes $x_i = 0$. Because the NTM branches at every variable, it effectively creates 2^n parallel computational paths—one for every possible combination of truth values. Each path then deterministically evaluates the formula using its specific assignment. If any path finds that the formula is 1, the entire NTM accepts.

   Notice that the "depth" of the computation tree is only n (the number of variables). Even though there are 2^n paths, the time taken along any single branch is polynomial.

   The DTM Approach (The Verifier) This is the most common way to prove SAT is in NP. We focus on how easy it is to check a potential answer. Suppose someone claims a formula is satisfiable. Their "certificate" (or witness) is a specific assignment of truth values (1 or 0) for every variable $x_1, x_2, \ldots, x_n$ in the formula. Substitute the given truth values into the formula. Evaluate each clause. Check if the entire expression evaluates to 1.

   If the formula has m clauses and n variables, a deterministic machine can evaluate the expression in $O(n + m)$ time. Since this is polynomial relative to the input size, SAT is in NP.

2. SAT $\in$ NP-HARD.
   See the excerpt from the book posted on the course web page (Cook-Levin-Theorem.pdf).

7 More NP-COMPLETE languages

Richard Karp, 1972.

7.1 3-SAT

3-SAT= $\{< \Phi > \mid \Phi$ is a Boolean formula in "3-CNF" that is satisfiable$\}$

Definition CNF: Conjunctive Normal Form. A list of OR clauses connected by AND, such as:

$$(\ldots \vee \ldots \vee \ldots) \wedge (\ldots \vee \ldots) \wedge (\ldots \vee \ldots \vee \ldots \vee \ldots)$$

3-CNF: each clause consists of 3 literals.

Example

$$\Phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_4 \vee x_5 \vee x_1)$$

Fact: Every Boolean formula can be converted into 3-CNF in polynomial time.

Theorem 7.1 3-SAT *is* NP-COMPLETE.

Proof The proof has two parts:

1. 3-SAT $\in$ NP.
   This is easy. An NTM can test every possible assignment; or, a DTM can verify a possible solution.
2. 3-SAT $\in$ NP-HARD.
   SAT $\leq_P$ 3-SAT by the fact given above. ∎

7.2 CLIQUE

A clique is a fully connected undirected graph.

2-clique

3-clique

4-clique

CLIQUE= $\{< G, k > \mid$ G contains a k-clique$\}$

Instance: A graph $G = (V, E)$ and a positive integer k
Question: Does G contain a k-clique?

Theorem 7.2 CLIQUE *is* NP-COMPLETE.

Proof The proof has two parts:

1. CLIQUE $\in$ NP.
   An NTM can test every possible subsets of k vertices to see whether it is a clique or not.
2. CLIQUE $\in$ NP-HARD.
   3-SAT $\leq_P$ CLIQUE as described below.

   A given 3-SAT instance can be converted into a CLIQUE instance as follows:
   Let $\Phi = C_1 \wedge C_2 \wedge \ldots \wedge C_k$. Generate a graph $G = (V, E)$:

   - Generate a vertex for each literal of each clause C_i.
   - Draw an edge between two vertices x and y if:

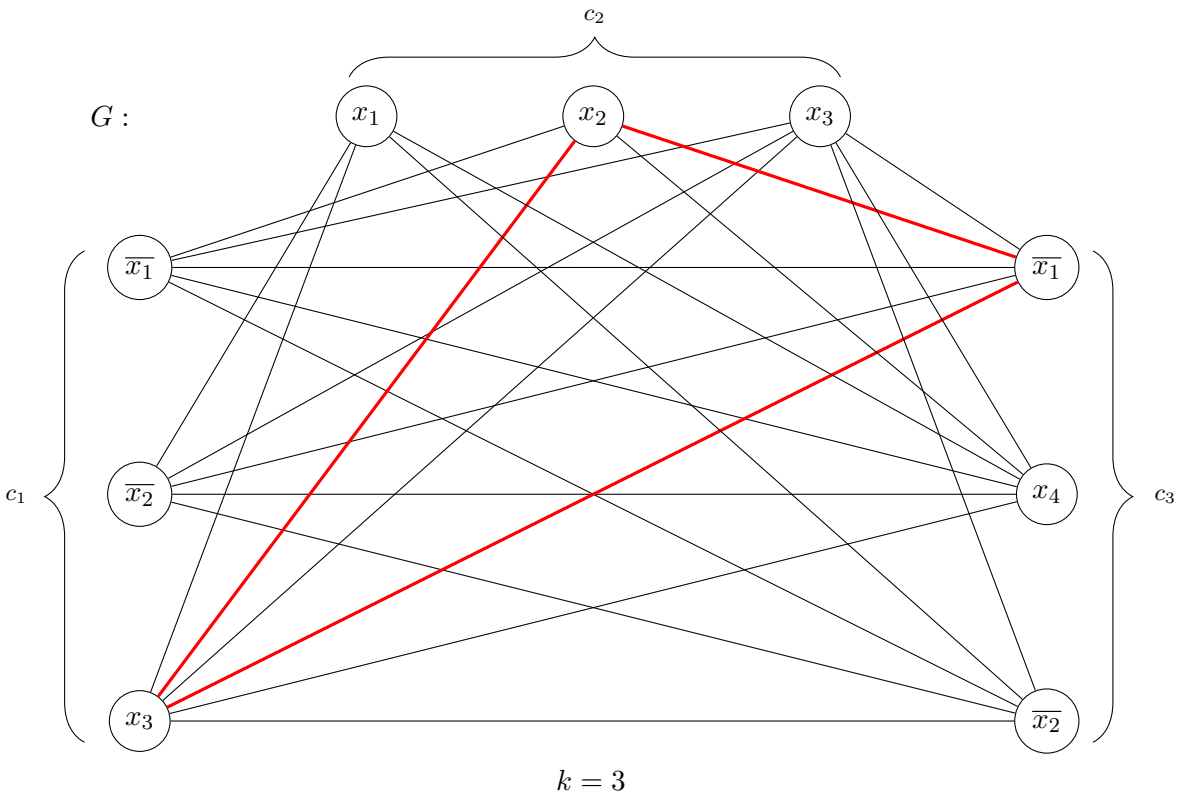
- x and y are not in the same clause.
- x and y are not contradictory (i.e., not like $x = x_1$). $y = \overline{x_1}$.

Also take k = number of clauses in Φ.
G has a k-clique if and only if Φ has a satisfying assignment:

- If G has a k-clique, that clique must have one vertex from each clause that are not contradictory. If we set these literals to 1 in Φ, it will be a satisfying assignment for Φ.
- If Φ has a satisfying assignment, that assignment must contain at least one literal from each clause set to 1. If we choose one such literal from each clause and take its vertex, they will make a k-clique in G.

Hence, 3-SAT $\leq_P$ CLIQUE. Therefore, CLIQUE is NP-COMPLETE. ∎

Example Let $\Phi = \underbrace{(\overline{x_1} \vee \overline{x_2} \vee x_3)}_{c_1} \wedge \underbrace{(x_1 \vee x_2 \vee x_3)}_{c_2} \wedge \underbrace{(\overline{x_1} \vee x_4 \vee \overline{x_2})}_{c_3}$

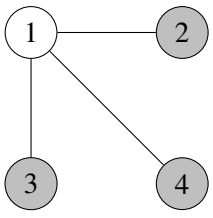


7.3 INDEPENDENT-SET

Definition Independent set is a subset of vertices in a graph with no edges between them.

Example Given the following graph

G:



{2,3,4} is an independent set in G.

INDEPENDENT-SET= $\{< G, k > \mid$ G contains k independent vertices$\}$

Instance: A graph $G = (V, E)$ and a positive integer k
Question: Does G contain k independent vertices?

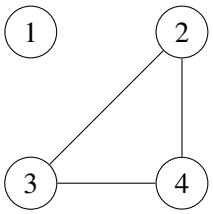
Theorem 7.3 INDEPENDENT-SET *is* NP-COMPLETE.

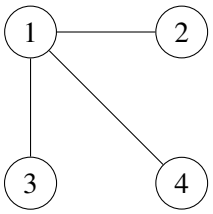
Proof The proof has two parts:

1. INDEPENDENT-SET $\in$ NP.
   An NTM can test every possible subset of k vertices to see whether they are independent or not.

2. INDEPENDENT-SET $\in$ NP-HARD.
   CLIQUE $\leq_P$ INDEPENDENT-SET as described below.

For a given graph $G = (V, E)$, consider its complement $G' = (V, \overline{E})$ where an edge (x, y) is in $\overline{E}$ if and only if it is not in E. For example:

G: 

G': 

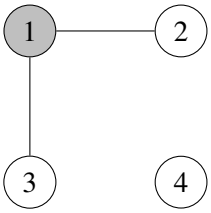
Claim: A vertex subset $V' \subseteq V$ is a clique in G if and only if it is an independent set in G'.

Regarding CLIQUE $\leq_P$ INDEPENDENT-SET, given a CLIQUE instance $G = (V, E)$ and k, generate an INDEPENDENT-SET instance $G' = (V, \overline{E})$ and k. G' has an independent set of size k if and only if G has a k-clique. Furthermore, they must be the same set of vertices V'. Therefore, INDEPENDENT-SET is NP-COMPLETE. ∎

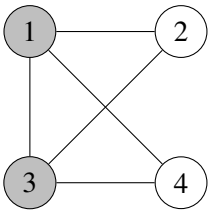
7.4 VERTEX-COVER

Definition Vertex cover is a subset of vertices that "cover" all the edges.

Example See the following graphs

G_1: 

{1} is a a vertex cover.

G_2: 

{1,3} is a a vertex cover.

VERTEX-COVER= $\{< G, k > \mid$ G has a vertex cover with k vertices$\}$

Instance: A graph $G = (V, E)$ and a positive integer k
Question: Does G contain a cover with k vertices?

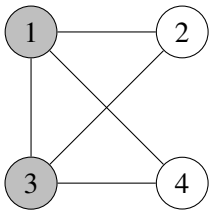
Theorem 7.4 VERTEX-COVER *is* NP-COMPLETE.

Proof The proof has two parts:

1. VERTEX-COVER $\in$ NP.
   An NTM can test every possible subsets of k vertices to see whether they cover all edges or not.

2. VERTEX-COVER $\in$ NP-HARD.
   INDEPENDENT-SET $\leq_P$ VERTEX-COVER as described below.

Observe that $V' \subseteq V$ is a vertex cover if and only if $V - V'$ is an independent set. For example:

G: 

{1,3} is a a vertex cover. {2,4} is an independent set.

Regarding INDEPENDENT-SET $\leq_P$ VERTEX-COVER, for a given independent set instance $< G, k >$, generate the vertex cover instance $< G, n - k >$, where $n = |V|$. G has k independent vertices if and only if G has a vertex cover of size $n - k$. Therefore VERTEX-COVER is NP-Complete. ∎

7.5 SUBSET-SUM

Instance: a set of positive integer weights (not necessarily distinct) and a positive integer "target" t:

$$S = \{w_1, w_2, \ldots, w_k\} \text{ and target } t$$

Question: Does $S' \subseteq S$ exist such that:

$$\sum_{w \in S'} w = t$$

Theorem 7.5 SUBSET-SUM *is* NP-COMPLETE.

Proof The proof has two parts:

1. SUBSET-SUM $\in$ NP.
   An NTM can test every possible subset of S to see whether their summation equals the target or not.

2. SUBSET-SUM $\in$ NP-HARD.
   3-SAT $\leq_P$ SUBSET-SUM as described below.

Given a 3-SAT instance Φ with n variables $x_1, x_2, \ldots, x_n$ and k clauses $C_1, C_2, \ldots, C_k$, generate a SUBSET-SUM problem with a set S with $2n + 2k$ elements:

- For each variable x_i, generate two elements y_i and z_i in S - one for x_i and one for $\overline{x_i}$.
- For each clause C_i, generate two variables g_i and h_i.

Each of these elements will be $(n + k)$-digit integers with digits $(d_1 d_2 d_3 \ldots d_n e_1 e_2 \ldots e_k)$. These integers are defined as follows:

- y_i and z_i has the digit d_i set to 1. Also if x_i appears in C_j, y_i has e_j set to 1. Similarly if $\overline{x_i}$ appears in C_j, z_i has e_j set to 1.
- g_i and h_i have the digit e_i set to 1.
- All remaining digits are 0.

The target integer t has:

- $d_i = 1$ for all d_i.
- $e_i = 3$ for all e_i.

Example

$$\Phi = (\overline{x_1} \vee \overline{x_2} \vee x_3) \wedge (x_1 \vee x_2 \vee x_3) \wedge (\overline{x_1} \vee x_4 \vee \overline{x_2})$$

	d_1	d_2	d_3	d_4	e_1	e_2	e_3
y_1	1	0	0	0	0	1	0
z_1	1	0	0	0	1	0	1
y_2		1	0	0	0	1	0
z_2		1	0	0	1	0	1
y_3			1	0	1	1	0
z_3			1	0	0	0	0
y_4				1	0	0	1
z_4				1	0	0	0
g_1					1	0	0
h_1					1	0	0
g_2						1	0
h_2						1	0
g_3							1
h_3							1
t	1	1	1	1	3	3	3

Let $S' = \{z_1, y_2, y_3, y_4, g_1, g_2, g_3\}$.

$$\sum_{w \in S'} w = 1,000,101 + 100,010 + 10,110 + 1,001 + 100 + 10 + 1 = 1,111,333$$

This corresponds to:

$$\overline{x_1} = x_2 = x_3 = x_4 = 1$$

which satisfies Φ.

$S' \subseteq S$ with weight t exists if and only if Φ has a satisfying assignment. ∎

7.6 PARTITION

Instance: $S = \{w_1, w_2, ..., w_k\}$

Question: Does $S' \subseteq S$ exist such that:

$$\sum_{w \in S'} w = \sum_{w \notin S'} w$$

Theorem 7.6 PARTITION *is* NP-COMPLETE.

Proof The proof has two parts:

1. PARTITION $\in$ NP
   An NTM can test all possible subsets of S to check whether it is a partition or not.
2. PARTITION $\in$ NP-HARD
   SUBSET-SUM $\leq_p$ PARTITION as described below.

Example Given a SUBSET-SUM problem $S = \{1, 3, 5\}$ and $t = 6$, generate a new set $S_2 = \{1, 3, 5, 3\}$. Here we add 3 since $3 = 2 \cdot t - W$, where W is the total weight of elements in S).

S_2 has an equal partition:

$$S_2' = \{1, 5\},\ S_2'' = \{3, 3\}$$

The subset without the new element is the $S' \subseteq S$ with weight $t = 6$.

How about $t < \frac{w}{2}$?

Example Given $S = \{1, 3, 5\}$ and $t = 4$, we generate:

$$S_2 = \{1, 3, 5, 1\}\ (1 = W - 2 \cdot t)$$

If S_2 has an equal partition, take the subset with the new element and remove it:

$$S_2' = \{1, 3, 1\} \text{ and } S_2'' = \{5\}$$

So, to show SUBSET-SUM $\leq_p$ PARTITION, given a SUBSET-SUM instance:

$$S = \{w_1, w_2, ..., w_k\}, \text{ and target } t,$$

generate the PARTITION instance as

$$S_2 = \{w_1, w_2, ..., w_k, w_{k+1}\}$$

where $w_{k+1} = |2t - W|$ and W denotes the total weight of S. ∎

Claim: S_2 has an equal partition $\iff$ S has subset with total weight t.

7.7 KNAPSACK

Instance: $S = \{x_1, x_2, ..., x_k\}$ within a *weight function* $w : S \to \mathbb{N}^+$, a *value function* $v : S \to \mathbb{N}^+$ a *weight limit* W, a *value limit* V.

Question: Does $S' \subseteq S$ exist such that

$$\sum_{x \in S'} w(x) \leq W, \sum_{x \in S'} v(x) \geq V$$

Theorem 7.7 KNAPSACK *is* NP-COMPLETE.

Proof The proof has two parts:

1. KNAPSACK $\in$ NP
   An NTM can pick an arbitrary subset S' and check whether it satisfies the weight and value limits in polynomial time.

2. KNAPSACK $\in$ NP-HARD
   SUBSET-SUM $\leq_p$ KNAPSACK as described below.

For a given SUBSET-SUM instance $S = \{w_1, w_2, ..., w_k\}$ and t, construct the KNAPSACK instance $S_2 = \{x_1, x_2, ..., x_k\}$ with $w(x_i) = v(x_i) = w_i$ and $W = V = t$.

Obviously, S_2 has a subset that satisfies the weight and value limits $\iff$ S has a subset S' with total weight equal to t. ∎

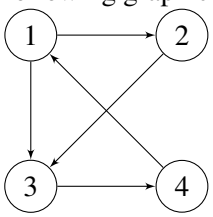
7.8 DIRECTED HAMILTONIAN CIRCUIT

Definition A Directed Hamiltonian Circuit in a directed graph is a cycle that visits every *vertex* in the graph (*exactly once*).

Instance: A directed graph $G = (V, E)$

Question: Does G contain a Directed Hamiltonian Circuit?

Example The following graph contains a Directed Hamiltonian Circuit.

G: 

$1 \to 2 \to 3 \to 4 \to 1$

Theorem 7.8 DIRECTED-HAMILTONIAN-CIRCUIT *is* NP-COMPLETE.

Proof The proof has two parts:

1. DIRECTED-HAMILTONIAN-CIRCUIT $\in$ NP
   An NTM can try every possible cycle starting from any node to check whether it is a Directed Hamiltonian Circuit or not.

2. DIRECTED-HAMILTONIAN-CIRCUIT is NP-HARD
   3-SAT $\leq_P$ DIRECTED-HAMILTONIAN-CIRCUIT (see the book) ∎

7.9 HAMILTONIAN-CIRCUIT

Instance: A graph $G = (V, E)$ (**Note:** Unless specified, a graph is assumed to be undirected)

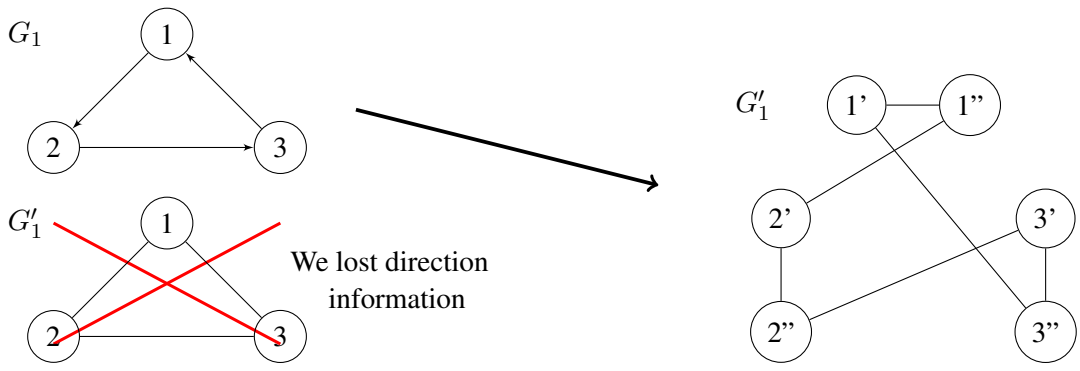
Question: Does G contain a HAMILTONIAN-CIRCUIT?

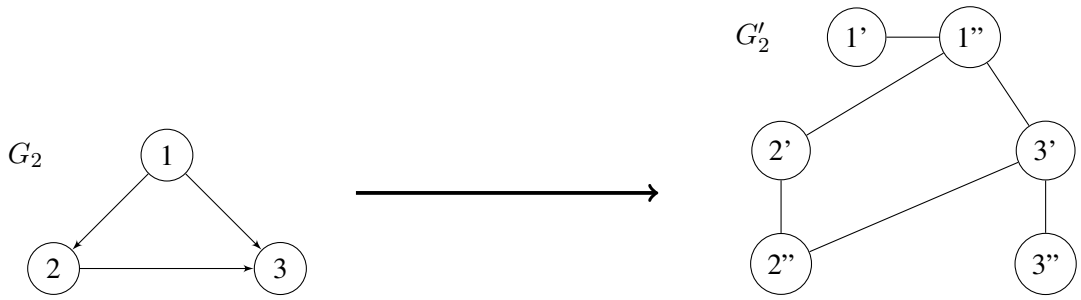
Theorem 7.9 HAMILTONIAN-CIRCUIT *is* NP-COMPLETE.

Proof The proof has two parts:

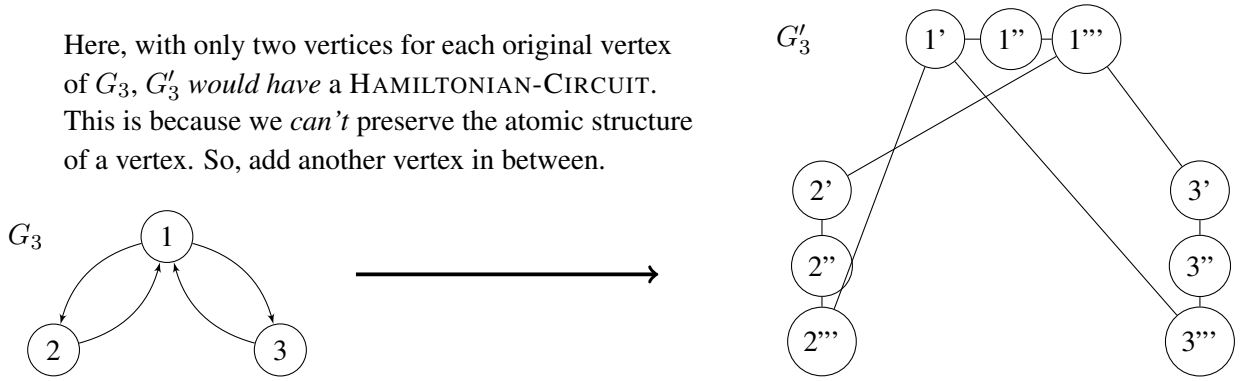
1. HAMILTONIAN-CIRCUIT $\in$ NP
   An NTM can try every possible cycle in the graph to check whether it is a Hamiltonian Circuit.

2. HAMILTONIAN-CIRCUIT $\in$ NP-HARD
   DIRECTED-HAMILTONIAN-CIRCUIT $\leq_P$ HAMILTONIAN-CIRCUIT as described below.





Here, with only two vertices for each original vertex of G_3, G'_3 *would have* a HAMILTONIAN-CIRCUIT. This is because we *can't* preserve the atomic structure of a vertex. So, add another vertex in between.



For a given DIRECTED-HAMILTONIAN-CIRCUIT instance, directed graph $G = (V, E)$, construct an undirected graph $G' = (V', E')$ as follows:

$$V' = \{v', v'', v''' \mid v \in V\}$$

$$E' = \{(v', v''), (v'', v''') \mid v \in V\} \cup \{(v''', u') \mid (v, u) \in E\}$$

Claim: G has a DIRECTED-HAMILTONIAN-CIRCUIT $\iff$ G' has a HAMILTONIAN-CIRCUIT.

Proof Exercise (see the book). ∎

7.10 TRAVELING SALESMAN PROBLEM

Instance: A graph $G = (V, E)$ with a weight (distance function) $w : E \to \mathbb{N}^+$, distance limit k.

Question: Does G contain a traveling salesman cycle with a total distance (weight) no more than k?

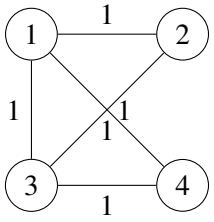
Theorem 7.10 TRAVELING-SALESMAN-PROBLEM *is* NP-COMPLETE.

Proof The proof has two parts:

1. TRAVELING-SALESMAN-PROBLEM $\in$ NP
   An NTM can generate an arbitrary ordering of the vertices and check whether that makes a traveling salesman tour with total weight k or less.

2. TRAVELING-SALESMAN-PROBLEM $\in$ NP-HARD
   HAMILTONIAN-CIRCUIT $\leq_P$ TRAVELING-SALESMAN-PROBLEM as described below.

We can convert any HAMILTONIAN-CIRCUIT into TRAVELING-SALESMAN-PROBLEM as follows:



$k = 4\ (= |V|)$

For a given HAMILTONIAN-CIRCUIT instance $G = (V, E)$ make it a weighted graph G' by assigning each edge the unit weight 1 and set $k = |v|$.

Obviously, G has a HAMILTONIAN-CIRCUIT $\Longleftrightarrow$ G' has a TRAVELING-SALESMAN-PROBLEM tour with total weight $|w|$. ∎